```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
import urllib.request

image_url_1 = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/box.png"
image_url_2 = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/box_in_scene.png"

urllib.request.urlretrieve(image_url_1, "image1.png")
urllib.request.urlretrieve(image_url_2, "image2.png")

print("Images downloaded successfully!")
```

```
Images downloaded successfully!
```

```
image1 = cv2.imread("image1.png")
image2 = cv2.imread("image2.png")

image1_rgb = cv2.cvtColor(image1, cv2.COLOR_BGR2RGB)
image2_rgb = cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)

print("Image 1 shape:", image1.shape)
print("Image 2 shape:", image2.shape)
```

```
Image 1 shape: (223, 324, 3)
Image 2 shape: (384, 512, 3)
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image1_rgb)
plt.title("Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(image2_rgb)
plt.title("Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

print("Images converted to grayscale successfully!")
```

```
Images converted to grayscale successfully!
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap="gray")
plt.title("Grayscale Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap="gray")
plt.title("Grayscale Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Histogram equalization to improve contrast
processed1 = cv2.equalizeHist(gray1)
processed2 = cv2.equalizeHist(gray2)
```

```
print("Basic preprocessing completed!")
```

```
Basic preprocessing completed!
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(processed1, cmap="gray")
plt.title("Preprocessed Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(processed2, cmap="gray")
plt.title("Preprocessed Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
sift = cv2.SIFT_create()

print("SIFT detector created successfully!")
```

```
SIFT detector created successfully!
```

```
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Image 1:")
print("Number of keypoints:", len(keypoints1))
print("Descriptor shape:", descriptors1.shape)

print("\nImage 2:")
print("Number of keypoints:", len(keypoints2))
print("Descriptor shape:", descriptors2.shape)
```

```
Image 1:
Number of keypoints: 604
Descriptor shape: (604, 128)

Image 2:
Number of keypoints: 969
Descriptor shape: (969, 128)
```

```
sift_image1 = cv2.drawKeypoints(
    image1_rgb,
    keypoints1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

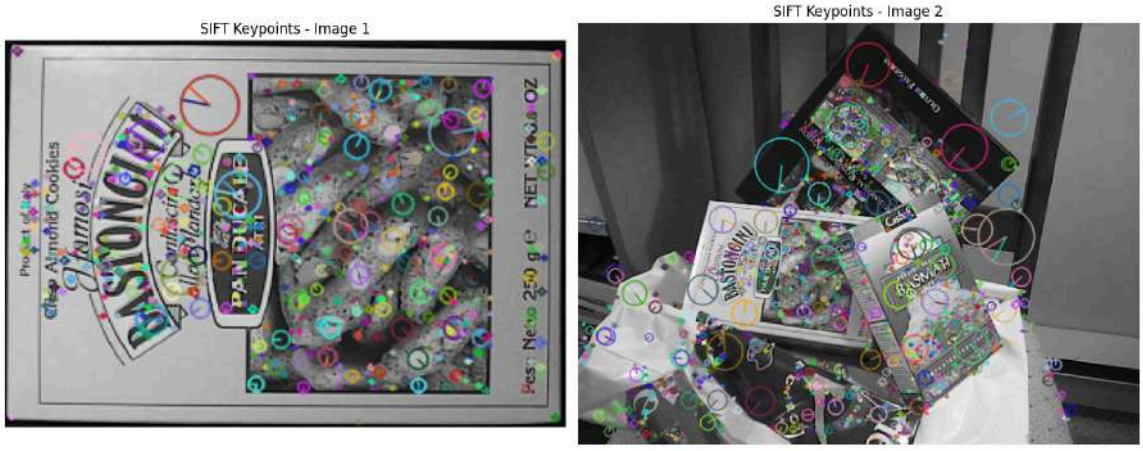
sift_image2 = cv2.drawKeypoints(
    image2_rgb,
    keypoints2,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.imshow(sift_image1)
plt.title("SIFT Keypoints - Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(sift_image2)
plt.title("SIFT Keypoints - Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
x1 = [kp.pt[0] for kp in keypoints1]
```

```
y1 = [kp.pt[1] for kp in keypoints1]

plt.figure(figsize=(8, 6))

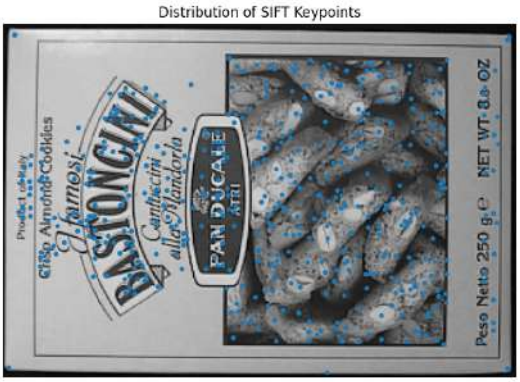
plt.imshow(image1_rgb)

plt.scatter(
    x1,
    y1,
    s=10
)

plt.title("Distribution of SIFT Keypoints")
plt.axis("off")

plt.show()

print("Total SIFT keypoints:", len(keypoints1))
```




```
Total SIFT keypoints: 604
```

```
print("SIFT Descriptor Information")
print("--------------------------------")

print("Number of descriptors:", descriptors1.shape[0])
print("Descriptor dimensions:", descriptors1.shape[1])
print("Descriptor data type:", descriptors1.dtype)
```

```
SIFT Descriptor Information
--------------------------------
Number of descriptors: 604
Descriptor dimensions: 128
Descriptor data type: float32
```

```
# Convert image to floating-point format
gray1_float = gray1 / 255.0

hog_features, hog_image = hog(
    gray1_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)

print("HOG features extracted successfully!")
print("Number of HOG features:", len(hog_features))
```

```
HOG features extracted successfully!
Number of HOG features: 36504
```

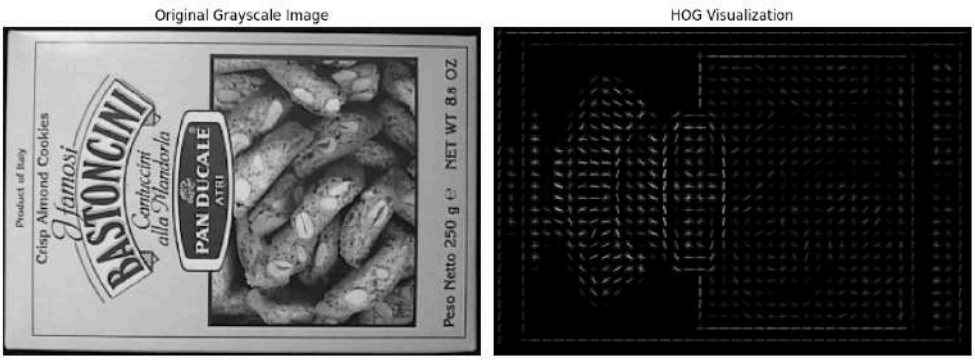
```
hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
gray2_float = gray2 / 255.0

hog_features2, hog_image2 = hog(
    gray2_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)
```

```
print("Image 2 HOG features:", len(hog_features2))
```

```
Image 2 HOG features: 106596
```

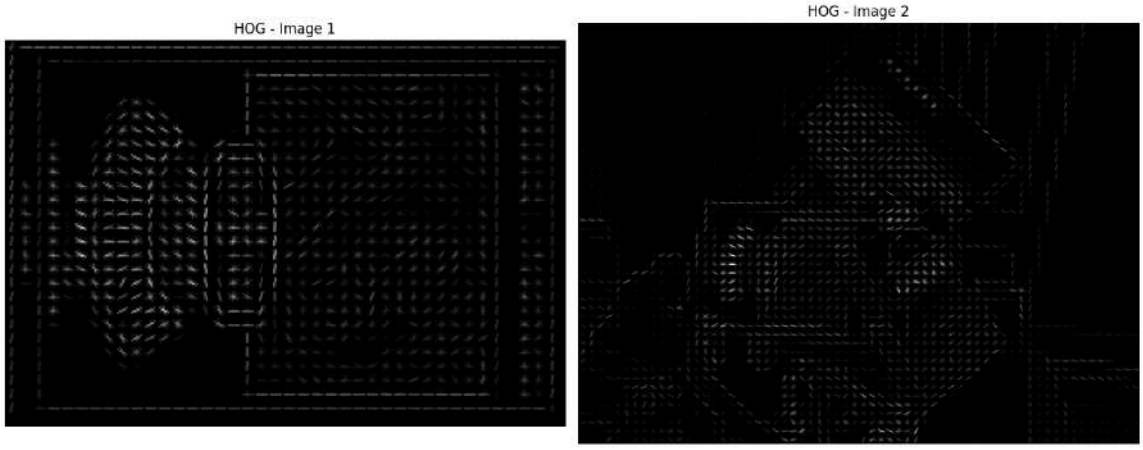
```
hog_image2_rescaled = exposure.rescale_intensity(
    hog_image2,
    in_range=(0, 10)
)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.imshow(exposure.rescale_intensity(hog_image, in_range=(0, 10)), cmap="gray")
plt.title("HOG - Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image2_rescaled, cmap="gray")
plt.title("HOG - Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
print("========== FEATURE COMPARISON ==========")

print("SIFT:")
print("Number of keypoints:", len(keypoints1))
print("Descriptor dimensions:", descriptors1.shape)

print("\nHOG:")
print("Number of features:", len(hog_features))
print("Feature vector dimensions:", hog_features.shape)

print("========================================")
```

```
========== FEATURE COMPARISON ==========
SIFT:
Number of keypoints: 604
Descriptor dimensions: (604, 128)

HOG:
Number of features: 36504
Feature vector dimensions: (36504,)
========================================
```

```
bf = cv2.BFMatcher()

matches = bf.knnMatch(
    descriptors1,
    descriptors2,
    k=2
)

print("Total candidate matches:", len(matches))
```

```
Total candidate matches: 604
```

```
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good matches:", len(good_matches))
```

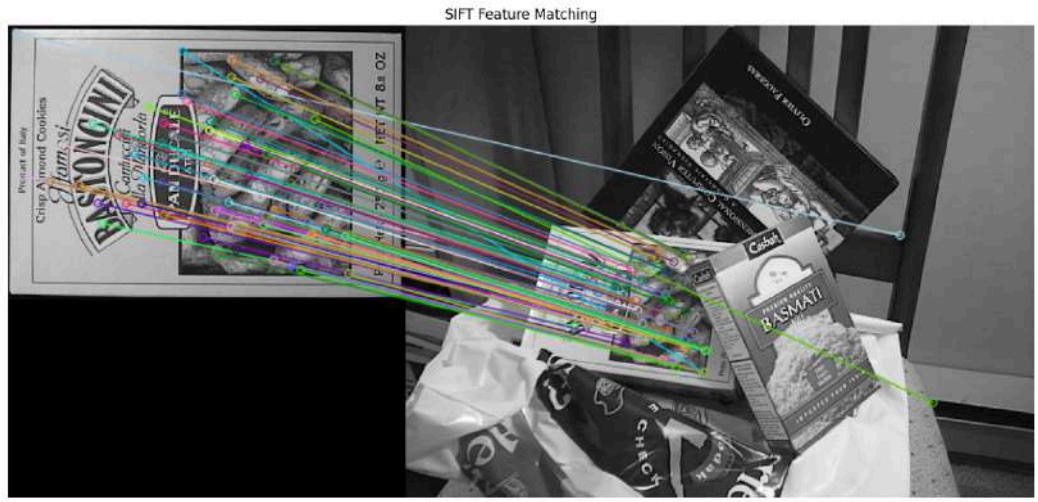
```
Good matches: 80
```

```
matched_image = cv2.drawMatches(
    image1_rgb,
    keypoints1,
    image2_rgb,
    keypoints2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))

plt.imshow(matched_image)
plt.title("SIFT Feature Matching")
plt.axis("off")

plt.show()
```




```
matching_percentage = (
    len(good_matches) /
    min(len(keypoints1), len(keypoints2))
) * 100

print(f"Matching Percentage: {matching_percentage:.2f}%")
```

```
Matching Percentage: 13.25%
```

```
# Resize both images to the same size before extracting HOG
hog_size = (256, 256)

resize1 = cv2.resize(gray1, hog_size)
resize2 = cv2.resize(gray2, hog_size)

# Convert to floating point
resize1 = resize1 / 255.0
resize2 = resize2 / 255.0

# Extract HOG features again
hog_features, hog_image = hog(
    resize1,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)

hog_features2, hog_image2 = hog(
    resize2,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)

# Compare HOG vectors
hog_distance = np.linalg.norm(
    hog_features - hog_features2
)

print("HOG Feature Vector 1 Shape:", hog_features.shape)
print("HOG Feature Vector 2 Shape:", hog_features2.shape)
print("HOG Euclidean Distance:", hog_distance)
```

```
HOG Feature Vector 1 Shape: (34596,)
HOG Feature Vector 2 Shape: (34596,)
HOG Euclidean Distance: 30.132075375294857
```

```
print("================================================")
print("             SIFT vs HOG")
print("================================================")

print("""
SIFT:
- Detects local keypoints.
- Generates local feature descriptors.
- Robust to scale and rotation changes.
- Useful for feature matching and object recognition.

HOG:
- Represents local gradient orientations.
- Describes shape and edge structure.
- Commonly used for object detection.
- Computationally simpler for many fixed-shape detection tasks.

================================================
""")
```

```
================================================
             SIFT vs HOG
================================================

SIFT:
- Detects local keypoints.
- Generates local feature descriptors.
- Robust to scale and rotation changes.
- Useful for feature matching and object recognition.

HOG:
- Represents local gradient orientations.
- Describes shape and edge structure.
- Commonly used for object detection.
- Computationally simpler for many fixed-shape detection tasks.

================================================
```

```
print("SIFT Advantages:")
print("1. Strong scale invariance")
print("2. Strong rotation invariance")
print("3. Robust local feature representation")
print("4. Useful for image matching")

print("\nSIFT Limitations:")
print("1. Computationally more expensive than simpler descriptors")
```

```
print("2. Produces many keypoints and descriptors")
print("3. Less suitable for very large-scale real-time applications")
```

```
SIFT Advantages:
1. Strong scale invariance
2. Strong rotation invariance
3. Robust local feature representation
4. Useful for image matching

SIFT Limitations:
1. Computationally more expensive than simpler descriptors
2. Produces many keypoints and descriptors
3. Less suitable for very large-scale real-time applications
```

```
print("HOG Advantages:")
print("1. Captures shape and edge information")
print("2. Effective for object detection")
print("3. Relatively simple feature representation")
print("4. Useful for structured objects")

print("\nHOG Limitations:")
print("1. Sensitive to large changes in scale")
print("2. Sensitive to rotation")
print("3. Less effective for arbitrary object viewpoints")
print("4. Requires suitable cell and block parameters")
```

```
HOG Advantages:
1. Captures shape and edge information
2. Effective for object detection
3. Relatively simple feature representation
4. Useful for structured objects

HOG Limitations:
1. Sensitive to large changes in scale
2. Sensitive to rotation
3. Less effective for arbitrary object viewpoints
4. Requires suitable cell and block parameters
```

```
print("================================================")
print("                OBSERVATION")
print("================================================")

print("""
1. SIFT identifies distinctive local keypoints in the images.
2. SIFT descriptors can be used to match similar regions between images.
3. HOG captures gradient orientation and shape information.
4. HOG visualization highlights important edge and contour structures.
5. SIFT is particularly useful for image matching and object recognition.
6. HOG is particularly useful for shape-based object detection.
7. The choice between SIFT and HOG depends on the computer vision task.
""")
```

```
================================================
                OBSERVATION
================================================

1. SIFT identifies distinctive local keypoints in the images.
2. SIFT descriptors can be used to match similar regions between images.
3. HOG captures gradient orientation and shape information.
4. HOG visualization highlights important edge and contour structures.
5. SIFT is particularly useful for image matching and object recognition.
6. HOG is particularly useful for shape-based object detection.
7. The choice between SIFT and HOG depends on the computer vision task.
```

```
print("================================================")
print("                 CONCLUSION")
print("================================================")

print("""
SIFT and HOG feature extraction techniques were
successfully implemented using Python.

SIFT was used to detect keypoints and calculate
local feature descriptors. The detected keypoints
were visualized and used for feature matching.

HOG features were extracted to represent gradient
orientations and shape information in the image.
The HOG descriptor was also visualized.

SIFT and HOG were compared based on robustness,
feature representation, computational requirements,
and practical applications.

The experiment demonstrates that feature extraction
converts raw image information into meaningful
representations that can be used for image matching,
object recognition, and object detection.
""")
```

```
================================================
                 CONCLUSION
================================================

SIFT and HOG feature extraction techniques were
successfully implemented using Python.

SIFT was used to detect keypoints and calculate
local feature descriptors. The detected keypoints
were visualized and used for feature matching.

HOG features were extracted to represent gradient
orientations and shape information in the image.
The HOG descriptor was also visualized.

SIFT and HOG were compared based on robustness,
feature representation, computational requirements,
and practical applications.

The experiment demonstrates that feature extraction
converts raw image information into meaningful
representations that can be used for image matching,
object recognition, and object detection.
```

Q1. What is feature extraction, and why is it important in computer vision?

Ans - Feature extraction is the process of converting raw image data into meaningful and useful features or descriptors. These features represent important information such as edges, shapes, textures, and distinctive points in an image. Feature extraction is important because it reduces complex image information into a form that can be used for object recognition, image matching, tracking, and classification. Good feature descriptors should be robust to changes in scale, rotation, illumination, and viewpoint.

Q2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

Ans - SIFT is a feature extraction technique used to detect distinctive keypoints and generate feature descriptors for an image. It first detects important points at different scales, determines their orientation, and then creates descriptors representing the local image regions around those points. Because the features are detected across different scales and assigned orientations, SIFT can recognize the same object even when its size or rotation changes.

Q3. What are keypoints and feature descriptors in image analysis?

Ans - Keypoints are distinctive points or regions in an image that contain important visual information, such as corners, edges, or other unique patterns. Feature descriptors are numerical representations of the local image information surrounding these keypoints. Keypoints

identify important locations, while descriptors describe their visual characteristics so that they can be compared between images.

Q4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

Ans - Histogram of Oriented Gradients (HOG) is a feature extraction technique that represents an image using the directions and strengths of local image gradients. The image is divided into small cells, gradient orientations are calculated within each cell, and the information is grouped into histograms. These histograms are normalized using blocks to produce the final feature descriptor. HOG is significant because it effectively captures shape and edge information and is useful for object detection and recognition.

Q5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Ans - SIFT and HOG are both feature extraction techniques, but they represent images differently. SIFT focuses on distinctive local keypoints and descriptors and is highly robust to scale and rotation changes. HOG focuses mainly on gradient orientations and shape or edge structure. SIFT is generally more computationally complex and is commonly used for image matching, object recognition, and tracking, while HOG is commonly used for shape-based object detection and recognition. The choice between them depends on the requirements of the computer vision application.

Q6. Why is SIFT considered invariant to scale and rotation?

Ans - SIFT is considered scale invariant because it detects keypoints at multiple image scales, allowing the same feature to be identified even when the object appears larger or smaller. It is rotation invariant because an orientation is assigned to each keypoint based on the local image gradients. The descriptor is then constructed relative to this orientation, allowing the feature to remain recognizable even when the image is rotated.

Q7. Mention three real-world applications where HOG descriptors are commonly used.

Ans - Three real-world applications of HOG descriptors are pedestrian detection, vehicle detection, and object recognition. HOG is particularly useful in applications where the shape and edge structure of an object are important for identifying it.

Q8. Why is feature extraction performed before image classification or object detection?

Ans - Feature extraction is performed before image classification or object detection because raw images contain a large amount of complex pixel information. Feature extraction converts this raw information into meaningful descriptors that highlight important characteristics such as shapes, edges, textures, and distinctive regions. This makes the data more suitable for classification and detection algorithms and can improve computational efficiency and recognition performance.

Q9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Ans - Handcrafted feature descriptors such as SIFT and HOG are designed using predefined rules and mathematical techniques. Their advantages include relatively understandable feature representations, lower data requirements, and usefulness for specific computer vision tasks. However, they may not perform well when images contain large variations in appearance, viewpoint, or complex patterns. Deep learning methods can automatically learn complex features from large datasets and often provide better performance for difficult recognition tasks, but they generally require more computational resources and training data.

Q10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Ans - Feature extraction techniques convert important visual information from images into numerical feature representations that can be compared and analyzed. In image matching, descriptors from two images can be compared to find corresponding regions. In face recognition, important facial features can be extracted and compared with stored representations. In object detection, features such as edges, shapes, and local patterns help identify and locate objects within images. Thus, feature extraction provides the fundamental image representation required by many computer vision systems.